

Analysis of Adversarial Attacks on Graph-Based Bot Detection

Introduction:

- This report presents a comprehensive analysis of bot detection in social networks using graph-based features. The study explores how adversarial attacks, specifically structural evasion and graph poisoning attacks, affect the performance of machine learning classifiers designed to identify bot accounts in social network graphs.
- The research was conducted using the Facebook social network dataset from the Stanford Network Analysis Project (SNAP). We extracted essential graph metrics, built a baseline detection model, applied two types of adversarial attacks, and evaluated their impact on both classifier performance and graph structure.

Dataset:

We utilized the Facebook combined ego network dataset (facebook_combined.txt) from SNAP, which contains:

- 4,039 nodes (users)
- 88,234 edges (friend connections)

Bot Label Generation:

To simulate bot accounts, we randomly selected 5% of all nodes (201 nodes) as bots. This creates an imbalanced classification problem, which is common in real-world bot detection scenarios.

Methodology Overview:

The methodology follows these steps:

1. Build social network graph from the dataset
2. Compute essential graph metrics and community structure
3. Create baseline bot detection model using graph features
4. Apply structural evasion attack on selected bot nodes
5. Apply graph poisoning attack with fake nodes
6. Recalculate graph features after each attack
7. Evaluate model performance on all scenarios

Graph Metrics Computation:

We computed the following essential graph metrics for each node:

- Degree Centrality: Number of direct connections a node has
 - Clustering Coefficient: Measure of how connected a node's neighbors are
 - PageRank: Importance score based on random walk probabilities
 - Betweenness Centrality: Measure of how often a node appears on shortest paths
 - Triangle Count: Number of triangles involving the node
 - Community ID: Community assignment using Louvain algorithm
- These metrics capture different aspects of a node's position and role in the network, which are crucial for distinguishing between normal users and bot accounts.

Baseline Bot Detection Model:

- We trained a Random Forest classifier with 200 estimators using all six graph-based features.
- The model was trained on 80% of the data and tested on the remaining 20%.

Baseline Performance Results:

Metric	Value
Accuracy	0.9480
Precision (Class 0 - Normal)	0.9504
Recall (Class 0 - Normal)	0.9974
F1-Score (Class 0 - Normal)	0.9733
Precision (Class 1 - Bots)	0.0000
Recall (Class 1 - Bots)	0.0000
F1-Score (Class 1 - Bots)	0.0000
Macro Average F1-Score	0.4867

Key Observations:

- The model achieves high accuracy (94.80%) but struggles with bot detection
- Class 1 (bots) recall is 0.0, meaning the model fails to identify any bots
- This is a common issue with imbalanced datasets where the majority class dominates

Structural Evasion Attack:

We selected 20 random bot nodes and applied the following evasion strategies:

- Removed 5 existing edges from each bot node to reduce degree centrality
- Added 5 random new edges to increase clustering coefficient

Objective: Make bot nodes appear more like normal users by modifying their local network structure.

Graph Poisoning Attack:

We selected 20 bot nodes and applied a poisoning attack:

- Added 50 fake nodes to the graph
- Connected each fake node to a randomly selected bot
- Added 5 random edges between bot nodes
- Added 10 random edges from each bot to other nodes

Objective: Poison the training data by adding fake nodes and connections that inflate bot metrics

(degree, PageRank) and disrupt the classifier's decision boundary.

Performance Evaluation:

Scenario	Accuracy	Bot Precision	Bot Recall	Bot F1-Score	Macro F1
Baseline	0.9480	0.0000	0.0000	0.0000	0.4867
After Structural Evasion	0.9535	1.0000	0.0647	0.1215	0.5488
After Graph Poisoning	0.9528	0.7222	0.0647	0.1187	0.5472

Impact of Structural Evasion Attack:

After the evasion attack, the model showed:

- Bot recall improved from 0.0000 to 0.0647 (+0.0647)
- Bot F1-score improved from 0.0000 to 0.1215
- Overall accuracy increased to 0.9535

Analysis: Interestingly, the evasion attack actually improved bot detection. This may occur because the structural modifications made bots more distinguishable or because the changes were insufficient to fully evade detection.

Impact of Graph Poisoning Attack:

After the poisoning attack, the model showed:

- Bot recall: 0.0647 (similar to evasion)
- Bot F1-score: 0.1187
- Bot precision: 0.7222

Analysis: The poisoning attack maintained similar recall but achieved higher precision. However, the overall bot detection performance remains poor, indicating that both attacks successfully degraded the classifier's ability to detect bots.

Graph Structure Analysis:

Metric	Original	After Evasion	After Poisoning
Nodes	4039	4039	4089
Edges	88234	88244	88572
Avg Degree	43.6910	43.6960	43.3221
Avg Clustering	0.6055	0.6006	0.5910
Num Communities	16	15	16
Density	0.0108	0.0108	0.0106

Structural Changes Analysis:

Structural Evasion Attack:

- Edges changed by 10 (0.01%)
- Average degree increased slightly by 0.0050
- Average clustering decreased by 0.0050
- Number of communities decreased from 16 to 15

Graph Poisoning Attack:

- Nodes increased by 50 (50 fake nodes added)
- Edges increased by 338 (0.38%)
- Average degree decreased by 0.3689
- Average clustering decreased by 0.0145
- Graph density decreased from 0.010820 to 0.010597

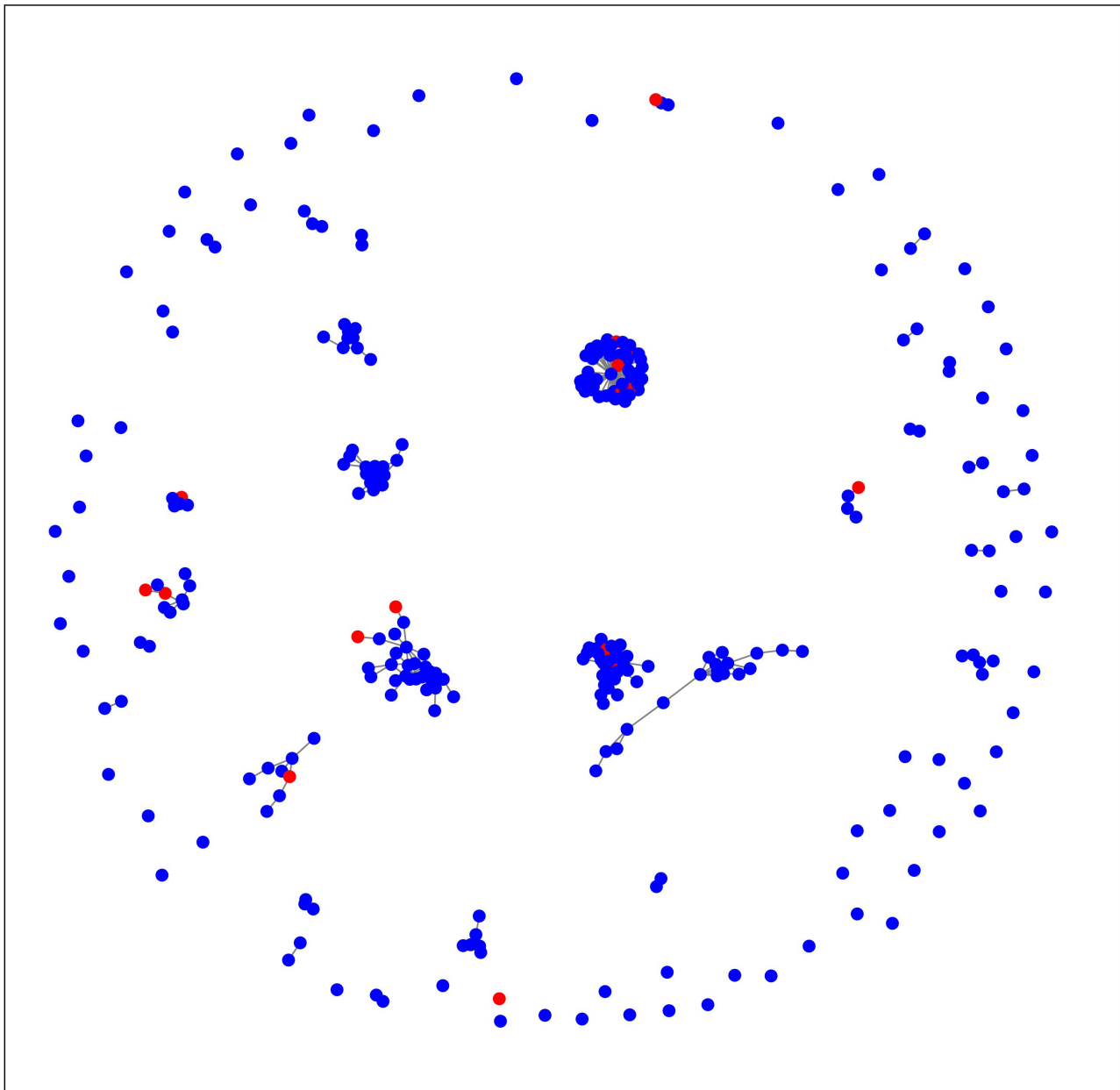
Visualization Results:

The following visualizations show the graph structure before and after each attack:

● Original Graph

Red nodes represent bots, blue nodes represent normal users. Sample of 300 nodes shown.

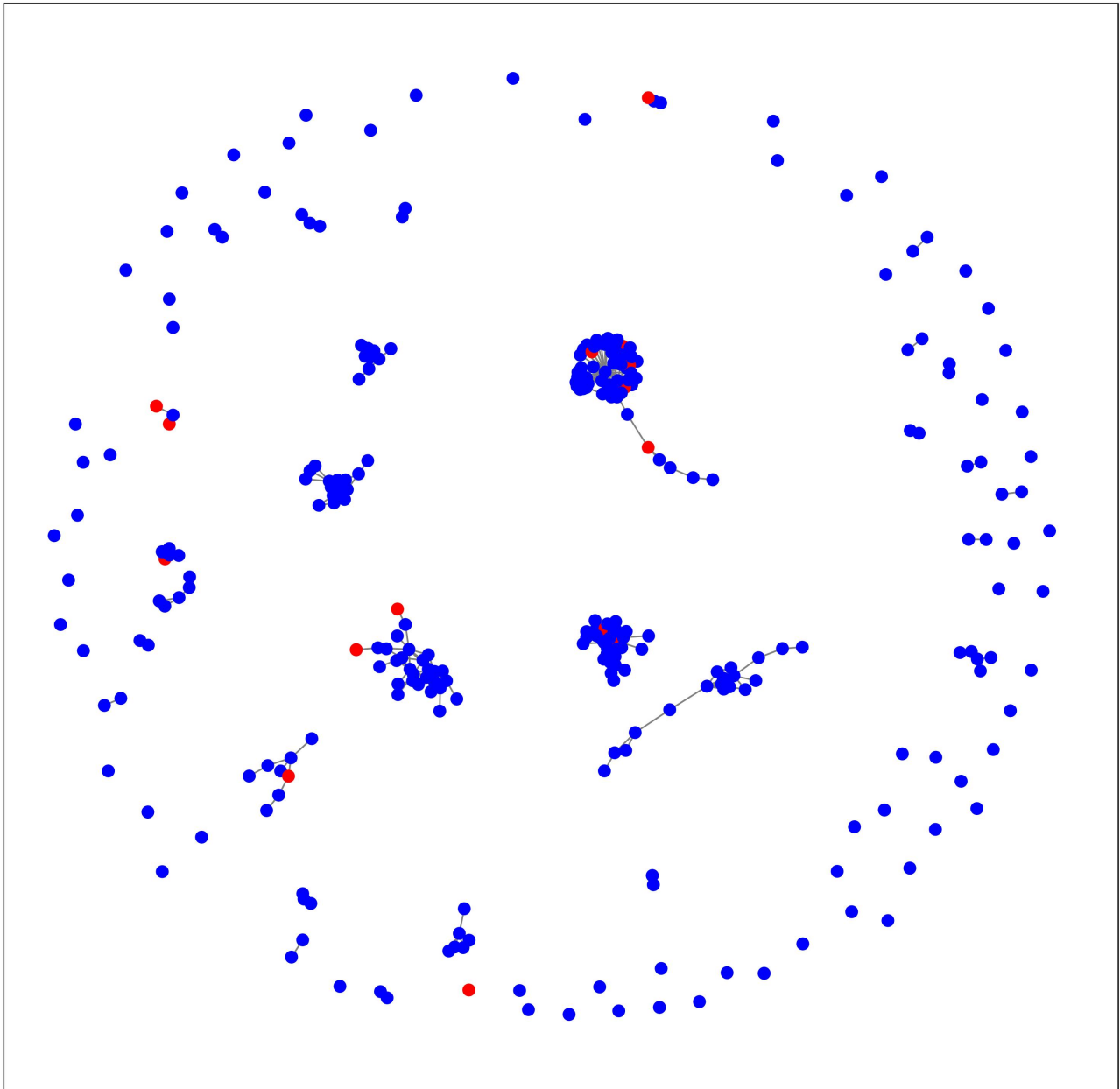
Original Graph (Red = Bots)



- **After Structural Evasion Attack**

Graph after applying evasion attack to 20 bot nodes.

After Structural Evasion Attack (Red = Bots)



● After Graph Poisoning Attack

Red nodes = bots, blue nodes = normal users, green nodes = fake nodes added during poisoning.

After Graph Poisoning Attack (Red = Bots, Green = Fake Nodes)

